Homework 2
Datalink Layer

Release Date 1404/2/4
Deadline 1404/2/26

Simulation Problems

In this part of assignment, you will work directly with the INET framework in OMNeT++ to simulate key data link layer concepts you've learned.

The INET Framework is an open-source model library for the OMNeT++ simulation environment, primarily used for simulating wired, wireless, mobile, and Internet protocols and networks. Built on top of OMNeT++, INET provides a rich set of protocols, devices, and tools that allow researchers and developers to design and analyze network behaviors under various conditions. The framework includes implementations of protocols such as TCP/IP, UDP, IPv4/IPv6, Ethernet, Wi-Fi, Mobile Ad-hoc Networks (MANETs), and more, making it highly suitable for studying complex network topologies and performance. INET also supports modeling of mobility, propagation, interference, and physical layer behavior, allowing detailed and realistic simulations.

Performing Measurements Using INET Framework (70 pts)

In this part, we simulate examples to demonstrate the most commonly used statistics utilizing the INET framework of OMNeT++. If your OMNeT++ already doesn't include the INET framework, first, you need to install it using this guide or the provided course material. We will follow this series of showcases from the INET documentation step by step to practice doing measurements using the INET framework. For each simulation, first calculate the desired parameter manually and then compare it with simulation result. Include the manual calculations in your report. For help with plotting and other statistics in Omnet++, follow this tutorial.

1) Measuring Channel Throughput:

The channel throughput is measured by observing the packets that are transmitted through the transmission medium over time. For both wired and wireless channels, the throughput is measured for any pair of communicating network interfaces, separately for both directions. Channel throughput is a statistic of transmitter modules, such as the `PacketTransmitter` in `EthernetPhyLayer`. Throughput is measured with a sliding window. By default, the window is 0.1s or 100 packets, whichever comes first. The window parameters, such as the window interval, are configurable from the `.ini` file, as `module.statistic.parameter`. Now we want to configure a simulation for a wired network that contains two `StandardHosts` as source and destination, connected via 100 Mbps Ethernet. The network is illustrated in Figure 1.

Set up the UDP traffic generator on your source hosts so that it emits 1200 byte packets at random intervals averaging 200 µs (i.e. `source.productionInterval` follows an exponential distribution). Then carry out these steps:

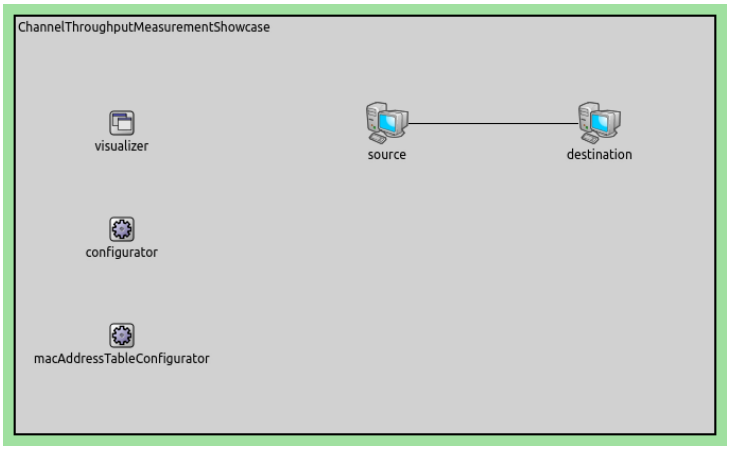


Figure 1: Network topology for performing measurements.

a) Build the network, run the simulation for 1 s, plot the observed throughput and compare it to your hand calculated value.

b) Repeat with packet sizes of 600 B and then 300 B. How does the throughput change? Contrast these results with those from step 1.

c) Now use 1800 byte packets. Observe how the frame transmitions change from previous parts. Justify this phenomenon (*Hint*: The MTU in Ethernet interfaces is 1500 Bytes).

d) Return to 1200 byte packets but set the mean interval to 400 μs. Report the new throughput and discuss how and why it has changed.

2) Measuring Channel Utilization:

The channel utilization statistic is measured by observing the packets that are transmitted through the transmission medium over time. For both wired and wireless channels, the utilization is measured for any pair of communicating network interfaces, separately for both directions. This statistic expresses the relative usage of the channel with a value between 0 and 1, where 0 means the channel is not used at all and 1 means the channel is fully utilized. Channel utilization is a statistic of transmitter modules, such as the `PacketTransmitter` in `EthernetPhyLayer`. The channel utilization is related to channel throughput in the sense that utilization is the ratio of throughput to channel datarate. By default, channel utilization is calculated for the past 0.1s or the last 100 packets, whichever comes first. These values are configurable from the `.ini` file with the `interval` (in unit of seconds) and `numValueLimit` parameters, as `module.statistic.parameter`.

For the model that you implemented in the last part, plot the utilization, and compare it with the result of your calculation.

3) Measuring End-to-end Delay:

The end-to-end delay is measured from the moment the packet leaves the source application to the moment the same packet arrives at the destination application. The end-to-end delay is measured by the `meanBitLifeTimePerPacket` statistic. The statistic measures the lifetime of the packet, i.e. time from creation in the source application to deletion in the destination application. The `meanBit` part

refers to the statistic being defined per bit, and the result is the mean of the per-bit values of all bits in the packet. When there is no packet streaming or fragmentation in the network, the bits of a packet travel together, so they have the same lifetime value.

a) To visualize the end-to-end delay, run the model that you implemented in the first part, and plot the `meanBitLifeTimePerPacket` statistic in both vector and histogram form.

b) Now consider that the source generates packets with a period of around 100 μs randomly (You can user normal distribution for this purpose). Again, plot the `meanBitLifeTimePerPacket` statistic in both vector and histogram form. Explain the results

c) Explain the differences between the results of parts (a) and (b).

4) Measuring Transmission Time:

The packet transmission time is measured from the moment the beginning of the physical signal encoding the packet leaves the network interface up to the moment the end of the same physical signal leaves the same network interface. This time usually equals the packet reception time that is measured at the receiver network interface from the beginning to the end of the physical signal. The exception would be when the receiver is moving relative to the transmitter at a relatively high speed compared to the propagation speed of the physical signal, but this is rarely the case in communication network simulation. For the model that you implemented, again consider that the source generates packets with a period of around 100 μs randomly.

In this part, the packet length is variable and follows a truncated normal distribution between 200 bytes and 800 bytes. Plot the source transmission time in both vector and histogram form. Explain the result and compare it with your calculation.

5) Measuring Data Rate:

The data rate is measured by observing the packets as are passing through overtime at a certain point in the node architecture. For example, an application source module produces packets over time and this process has its own data rate. Similarly, a queue module enqueues and dequeues packets over time and both of these processes have their own data rate. These data rates are different, which in turn causes the queue length to increase or decrease over time.

Consider that the source generates packets with a period of around 200 μs randomly. Run the simulation for 1 second, and report the source and destination data rates.

Visualizing Data Link Activity (30 pts)

Visualizing network traffic is an important aspect of running simulations. INET provides various visualizers to help you understand the network activity, including `DataLinkVisualizer` which focuses on the data link level. This module allows you to see the visual representation of the data link level traffic in the form of arrows that fade as the traffic ceases.

In this part, we will implement two simulation models of a showcase from the INET documentation step by step, to get more familiar with OMNeT++ and its features. For each section, add proper logs to your simulation and provide enough screenshots from your logs and simulation model to explain how the model is working properly.

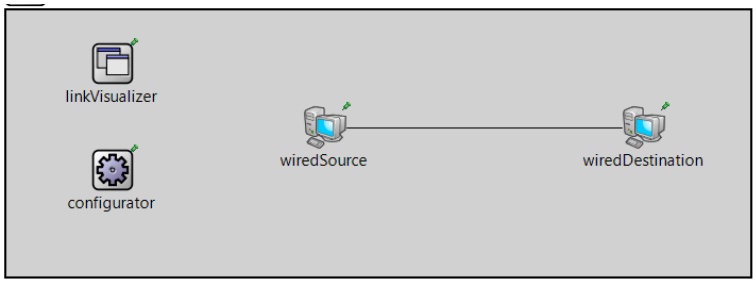


Figure 2: Simple network for visualizing Datalink activity.

1) Enabling Visualization of Data Link Activity:

In INET, data link activity can be visualized by including a `DataLinkVisualizer` module in the simulation. Adding an `IntegratedVisualizer` module is also an option because it also contains a `DataLinkVisualizer` module. Data link visualization is disabled by default; it can be enabled by setting the visualizer's `displayLinks` parameter to true.

Now we want to configure a simulation for a wired network that contains two `StandardHosts` as *wiredSource* and *wiredDestination*. The `linkVisualizer` module's type should be configured to be `DataLinkVisualizer`.

a) Create a network description file (`.ned`) as shown in Figure 2 that contains two `StandardHost` nodes, named *wiredSource* and *wiredDestination*, linked by a 100 Mb Ethernet connection. Add a `DataLinkVisualizer` module so the frames become visible, and include an `Ipv4NetworkConfigurator` to handle the upper-layer settings automatically.

b) In the `.ini` file, you should configure the simulator so that you can see the visualization. To do that, in your *wiredSource*, define the app to be the `pingApp` and the destination to be the *wiredDestination*. Also as mentioned, enable the `displayLinks` and set the `fadeOutTime` to be the most suitable to your eyes.

c) Now simulate the model and in analyze the datalink activity.

2) Filtering Data Link Activity:

In complex networks with many nodes and several protocols in use, it is often useful to be able to filter network traffic and visualize only the part of the traffic we are interested in. The following simulation shows how to set packet filtering in `DataLinkVisualizer`.

a) First, implement the network shown in Figure 3 with the knowledge you gained in the last part. For the Ethernet switch use `EthernetSwitch` and the links should still be 100 M ethernet.

b) In the simulation, the *source1* pings *destination1*, and the *source2* pings *destination2*. Adjust the application start times so that the run time of the apps don't collide with each other. For this network, the visualizer's type is `IntegratedVisualizer`. Set the data link activity visualization in a way that it is filtered to display only ping messages (*Hint*: use `packetFilter`, attribute to do that). Adjust the `fadeOutMode` and the `fadeOutTime` parameters so that the activity arrows do not fade out completely before the next ping messages are sent.

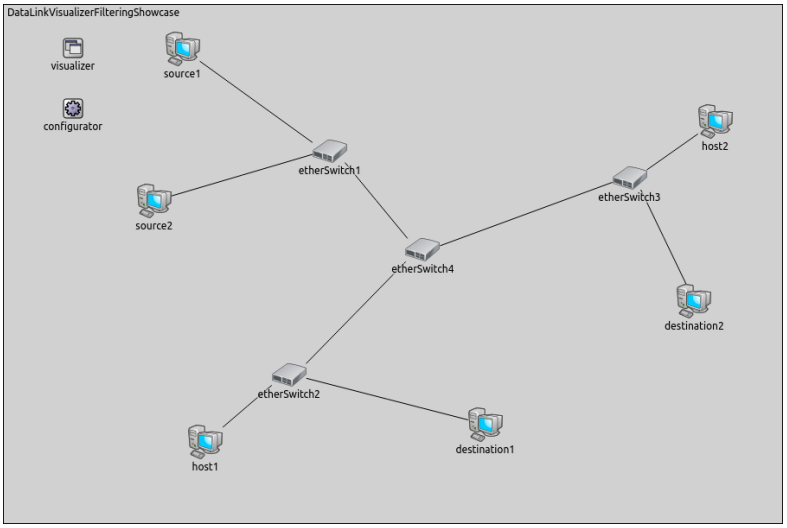


Figure 3: Network topology for visualizing Datalink activity.

c) Now, add *source3* and connect it to the *etherSwitch1*. Add *destination3* and connect it to the central Ethernet switch (*etherSwitch4*) using *etherSwitch5*. *text* pings *destination3*. One of the features of `dataLinkVisualizer` is filtering the traffic of specific nodes. Let's assume we want to display traffic between the hosts *source3* and *destination3* only, along the path *etherSwitch1*, *etherSwitch4*, and *etherSwitch5*. Set the visualizer's `nodeFilter` parameters to achieve this goal.

Submission

Submit all simulation files (.ini, .ned) along with your report, which should describe your simulation steps and answer all assigned questions. Package everything in a ZIP file named **DN_HW2_studentID.zip**, replacing `StudentID` with your actual student ID.